Chapter 16
Math libraries

Abstract Most of the open-source mathematics software is built upon libraries which implement mathematical functions and computations. Since mathematics is an integral part of most scientific computing and research, it is to be expected that these libraries can also often be used in application software written by the reader. In this chapter we discuss some of the most popular mathematical libraries which are used in open-source applications. The libraries cover almost all aspects of mathematics including linear algebra, number theory, graph theory, FFT, multi-precision computation and linear programming.

Contents

16.1 BLAS **350**
16.2 ATLAS **351**
16.3 LAPACK **351**
16.4 NTL **352**
16.5 GSL **352**
16.6 GMP **353**
16.7 MPFR **353**
16.8 FFTW **353**
16.9 GLPK **354**
16.10 COIN-OR: Comp. Infrastructure for OR **355**
16.11 Conclusion **356**

In this chapter we discuss several interesting and useful tools available for performing mathematics using computers. They include symbolic algebra, matrices, linear algebra, group theory, polynomials, statistics. Specifically we shall cover LAPACK, BLAS and ATLAS foundation libraries. NTL, number theory library, GNU Scientific library, GNU Multiprecision bignum library. For sparse solvers, TAUCS, FFT using FFTW library. Linear programming with glpk. Complete open sourced mathematical systems can be found in GNU Octave, Maxima, R, and Sage. Group theoretic software pari and its front-end 'gp'. A formalized mathematical system AXIOM and the REDUCE computer algebra software package. A front end to many of these software alongwith publication quality typesetting is 'TeXMacs'. We round-off this section with the discussion of the Computer Geometry Algorithms Library (CGAL).

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_16, © Springer Science+Business Media, LLC 2011

Linear algebra is significant component of almost all engineering and scientific work loads. In this section we discuss some of the software libraries which are available for linear algebra computation.

16.1 BLAS

Basic Linear Algebra Subprograms (BLAS) is an application programming interface (API) standard for publishing libraries to perform basic linear algebra operations such as vector and matrix multiplication. They are most commonly used in high-performance computing and are used to develop larger packages such as LAPACK. Optimized implementations of the BLAS interface have been developed by hardware vendors such as Intel and AMD as part of their MKL and ACML libraries respectively. BLAS is divided into 3 levels:

1. Level-1: vector only operations of the type $y \leftarrow \alpha x + y$,
2. Level-2: matrix-vector operations, $y \leftarrow \alpha Ax + \beta y$,
3. Level-3: matrix-matrix operations, such as: $C \leftarrow \alpha AB + \beta C$. This level contains the GMM (or general matrix multiply) operation.

The type of the vector or matrix is defined as part of the name of the operation; the type is one of single-float, double-float, or complex. As mentioned above many optimized implementation of BLAS are available, and since it is the building block of more complex libraries, performance improvement in BLAS are often critical to improving system performance. An example of a BLAS function to compute matrix-vector product is shown in Listing 16.1.

```
// \file blas_example.cpp
// \author Sandeep Koranne (C) 2010
// \description Example of using BLAS in C++
#include <iostream>           // Program IO
extern "C" {
#include <cblas.h>            // BLAS header
}
#if 0
cblas_dgemv(const enum CBLAS_ORDER Order,
      const enum CBLAS_TRANSPOSE TransA,
      const int M, const int N,
      const double alpha,
      const double *A, const int lda,
      const double *X, const int incX,
      const double beta,
      double *Y, const int incY);
#endif
static double X[] = { 1.0, 2.0, -2.0, -1.0 };
static double A[] = {
  4.0, 3.0, 1.0, 2.0,
  2.0, 1.0, 5.0, 3.0,
  6.0, 3.0, 1.0, 2.0,
  1.0, 4.0, 6.0, 2.0
};

int main( int argc, char* argv[] ) {
  std::cout << "A = ";
  for( int i=0; i < 4; ++i ) {
```

```
    for( int j=0; j < 4; ++j ) {
      std::cout << A[i*4+j] << " ";
    }
    std::cout << "\n";
  }
  double *y = new double[4];
  // compute Ax
  cblas_dgemv( CblasRowMajor, CblasNoTrans, 4, 4, 1.0,
         A, 4, X, 1, 0.0, y, 1 );
  for(int i=0; i < 4; ++i ) std::cout << y[i] << " ";
  std::cout << std::endl;
  return(0);
}
```

Listing 16.1 Example of using BLAS library

The use of the `cblas_dgemv` function demonstrates the flexibility of the underlying API.

16.2 ATLAS

ATLAS is an acronym for Automatically Tuned Linear Algebra Software. It provides an implementation of BLAS, but it is unique in the sense that it provides a completely automatic method of generating optimized BLAS libraries from the same code on any machine or architecture. Even though the performance of ATLAS cannot compare to hand-optimized machine specific software, ATLAS is often used to build a quick and optimized version of BLAS while the customized functions are implemented.

ATLAS uses automatic tuning of code parameters based on experimental runs of standardized software on the target machine as part of the compilation. Thus it is critical to use a representative machine (under nominal workload) to compile ATLAS. Given the speculative nature of modern architecture's instruction pipeline, and large dependency of data movement on cache, ATLAS uses data layout and cache line optimization to derive its performance.

When compiling code which uses the ATLAS libraries, the linking step should be performed in the following order:

```
-llapack -lcblas -lf77blas -latlas
```

This ordering is important for the linker to find the ATLAS library functions.

16.3 LAPACK

LAPACK (Linear Algebra PACKage) is a software library for numerical linear algebra. It provides routines for solving systems of linear equations and linear least squares, eigenvalue problems, and singular value decomposition. It also in-

cludes routines to implement the associated matrix factorizations such as LU, QR, Cholesky and Schur decomposition.

16.4 NTL

Number Theory Library (NTL) is a C++ library for performing experiments and computations in *number theory*. NTL supports arbitrary length integer computation, computation over finite fields, vectors, matrices, polynomials and lattices.

```
#include <NTL/ZZ.h>
NTL_CLIENT
int main() {
   ZZ a, b, c;
   cin >> a;
   cin >> b;
   c = (a+1)*(b+1);
   cout << c << "\n";
}
```

Listing 16.2 Example of using NTL

Running this example produces:

```
123456789
123456789
15241578997104100
```

16.5 GSL

GNU Scientific Library (GSL) is a software library for numerical computations in science and mathematics. The library implements the calculation of numerically evaluated functions such as Bessel functions. Consider the program shown in Listing 16.3.

```
#include <stdio.h>
#include <gsl/gsl_sf_bessel.h>
int main(void) {
  double x = 5.0;
  double y = gsl_sf_bessel_J0(x);
  printf("J0(%g) = %.18e\n", x, y);
  return 0;
}
```

Listing 16.3 Example of using GNU Scientific Library (GSL)

Running this program requires:

```
gcc $(gsl-config --cflags) gsl_example.c
    $(gsl-config --libs)
J0(5) = -1.77596771314338292e-01
```

The GSL library includes functions for:

- Physical constants and Basic mathematical functions
- Complex numbers, Polynomials, Vectors and matrices
- Special functions
- Permutations, Combinations and Multisets
- Linear algebra, Eigensystems
- FFT, Discrete Hankel transform and Discrete wavelet transform
- Numerical differentiation and integration
- Random number generation and Quasi-random sequences
- Statistics, Histograms, N-tuples
- Simulated annealing
- Ordinary differential equations
- Interpolation, Chebyshev approximations
- Root-finding in one and multiple dimensions
- Minimization in one and multiple dimensions
- Least-squares fitting, Nonlinear least-squares fitting
- IEEE floating-point arithmetic

16.6 GMP

GNU multi-precision library (GMP) is a library for arbitrary precision arithmetic on signed integers, rational numbers and floating point numbers. The only limit to the precision is imposed by the available memory of the machine. The applications of GMP include cryptography, computational geometry and computer algebra.

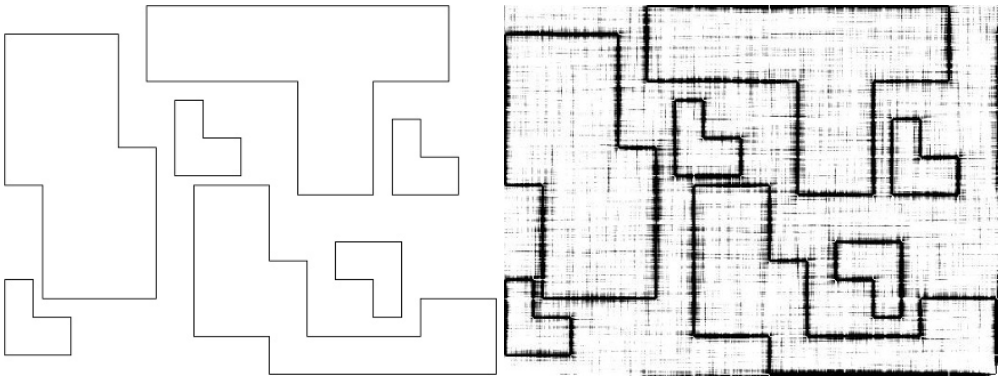
16.7 MPFR

GNU multi-precision float rounding (MPFR) is a library for arbitrary precision floating point computation with *correct rounding*. The computation is efficient and has correct mathematical semantics. It is used for computational geometry applications, and like GNU GMP, is now required to build GCC.

16.8 FFTW

FFTW is an acronym for "fastest Fourier transform in the west". It is a software library for computing discrete FFT and was developed at MIT. Using FFTW, transforms of real and complex values vectors can be carried out. It is one of the fastest FFT libraries. Like ATLAS, FFTW is based on a *plan* based computation, except in this case the user can define a plan at run time depending upon the type and size

of the vector. For a sufficiently large number of repeated transforms it is advantageous to use FFTW's ability to choose the fastest algorithm by actually measuring the performance of (some or all of) the supported algorithms on the given array size and platform. These measurements, which the authors call wisdom can be stored in a file or string for later use. A good example of the FFT can be shown using the GIMP fft filter:



(a) A collection of polygons transformed with FFT

Fig. 16.1 FFT transformations

The particular call to FFTW in this filter is shown below:

```
fftw_plan p;
p = fftw_plan_dft_r2c_2d(w, h, fft_real,
    (fftw_complex *) fft_real, FFTW_ESTIMATE);
fftw_execute(p);
```

This performs a real-to-complex in-place FFT on the data (assuming that the vector has memory for complex sized data).

16.9 GLPK

The GNU Linear Programming Kit (GLPK) is a software system designed to solve large-scale linear programming (LP), mixed integer programming (MIP) problems. The software comprises of a C language library which implements a revised simplex method, primal-dual method and Gomory's mixed integer cuts alongwith branch-and-bound algorithms. GLPK also supports the GNU MathProg modeling language which is a subset of AMPL. Consider the following GLPK sample code (which is available from the examples directory):

```
..
s1:   lp = glp_create_prob();
s2:   glp_set_prob_name(lp, "sample");
s3:   glp_set_obj_dir(lp, GLP_MAX);
```

```
s4:   glp_add_rows(lp, 3);
..
s11:  glp_add_cols(lp, 3);
s12:  glp_set_col_name(lp, 1, "x1");
s13:  glp_set_col_bnds(lp, 1, GLP_LO, 0.0, 0.0);
s14:  glp_set_obj_coef(lp, 1, 10.0);
..
s29:  ia[9] = 3, ja[9] = 3, ar[9] =  6.0; /* a[3,3] =  6 */
s30:  glp_load_matrix(lp, 9, ia, ja, ar);
s31:  glp_simplex(lp, NULL);
s32:  z = glp_get_obj_val(lp);
s33:  x1 = glp_get_col_prim(lp, 1);
```

The GLPK library has functions for defining the problem, specifying whether the objective function should be maximized or minimized, adding constraints and the objective function. We compile this program and run it as:

```
gcc sample.c -o sample -lglpk
* 0: obj =   0.000e+00 infeas = 0.0e+0 (0)
* 2: obj =   7.333e+02 infeas = 0.0e+0 (0)
OPTIMAL SOLUTION FOUND

z = 733.33; x1 = 33.33; x2 = 66.667; x3 = 0
```

16.10 COIN-OR: Comp. Infrastructure for OR

Another software system for linear programming, and much more, is the Computational Infrastructure for Operations Research (COIN-OR). It contains the following tools:

1. Developer tools: UNIX development tools, mainly developed for COIN-OR internal development,
   - BuildTools: CoinBazaar, CoinBinary, CoinWeb, and Coopr,
2. PFunc: Parallel Functions, API for task parallelism,
3. Graphs: tools and libraries for operating on graphs:
   - CGC: COIN-OR Graph Classes:
   - LEMON: Library of Efficient Models and Optimization in Networks, a C++ template library for combinatorial optimization,
4. Interfaces
   - CoinMP: API and library for CLP, CBC and CGL,
   - OSI: Open Solver Interface (see also Section 16.10.1,
   - PuLP: Python library for modeling LPs, and IPs.
5. Modeling Systems: Coopr, FlopC++, PuLP,
6. Optimizers
   - CLP: COIN-OR LP, a simplex solver,

- CoinMP: API and library for CLP, CBC and CGL,
- BCP: Branch-Cut-price framework,
- CBC: COIN-OR branch-and-cut, LP based branch-and-cut,
- CGL: Cut Generator Library,
- CHiPPS: COIN-OR High Performance Parallel Search framework,
- DIP: Decomposition in IP,
- SYMPHONY: callable library for MIPs,

16.10.1 Open Solver Interface

The COIN-OR Open Solver Interface is a standardized application programming interface (API) for calling solver libraries and external functions. It supports open-source as well as commercial solvers, such as COIN-OR LP, BCP, and GLPK.

16.11 Conclusion

A large fraction of engineering tools are based on a fixed set of mathematical libraries. The most important libraries which implement mathematical features are described in this chapter. In particular the mathematical libraries of BLAS, ATLAS, LAPACK, NTL, GSL, GMP and MPFR were discussed. The FFTW system for computing FFTs was also described with the help of an image-processing example. Linear programming has become an important solution method for engineering disciplines (especially financial applications). The GNU GLPK and COIN-OR systems for linear programming was discussed in Section 16.10.